# PostgreSQL Case Study

## Smart Retail Order Management System

## 1. Case Study Overview

A retail company named **SmartMart Retail Pvt. Ltd.** operates multiple stores across India. The company sells electronic products, home appliances, accessories, and office equipment.

Currently, customer, product, employee, and order information is maintained in separate spreadsheets. This creates several problems:

- Duplicate customer information
- Incorrect product stock
- Difficulty tracking orders
- Difficulty calculating sales by store
- No proper relationship between customers, orders, products, and employees
- Slow preparation of monthly sales reports
- Difficulty importing and exporting business data

SmartMart has decided to implement a PostgreSQL-based **Retail Order Management System**.

The database must manage:

- Stores
- Employees
- Customers
- Categories
- Products
- Orders
- Order items
- Payments

---

# 2. Learning Objectives

After completing this case study, participants should be able to:

- Create and manage PostgreSQL databases and tables
- Use primary keys and foreign keys
- Insert, update, and delete records
- Retrieve records using SELECT
- Filter records using WHERE
- Use DISTINCT, COUNT, BETWEEN, IN, and LIKE

- Group data using GROUP BY
- Filter grouped results using HAVING
- Use INNER, LEFT, RIGHT, FULL OUTER, and SELF JOIN
- Create aliases using AS
- Combine results using UNION
- Write subqueries
- Use mathematical, string, date, and timestamp functions
- Import and export data using PostgreSQL utilities
- Generate business reports from relational data

---

# 3. Database Design

The system will contain the following tables:

1. `stores`
2. `employees`
3. `customers`
4. `categories`
5. `products`
6. `orders`
7. `order_items`
8. `payments`

## Table Relationships

- One store can have many employees.
- One employee may report to another employee.
- One category can contain many products.
- One customer can place many orders.
- One employee can process many orders.
- One order can contain many products.
- One product can appear in many orders.
- One order can have one or more payment records.

---

# 4. Database Creation

```
CREATE DATABASE smartmart_db;
```

Connect to the database:

```
\c smartmart_db
```

---

# 5. Table Creation

## 5.1 Stores Table

```
CREATE TABLE stores (
    store_id SERIAL PRIMARY KEY,
    store_name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL,
    state VARCHAR(50) NOT NULL,
    opening_date DATE,
    active BOOLEAN DEFAULT TRUE
);
```

## 5.2 Employees Table

The `manager_id` column creates a self-referencing relationship.

```
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    employee_name VARCHAR(100) NOT NULL,
    email VARCHAR(120) UNIQUE,
    designation VARCHAR(60),
    salary NUMERIC(10,2) CHECK (salary > 0),
    joining_date DATE,
    store_id INTEGER,
    manager_id INTEGER,
    FOREIGN KEY (store_id)
        REFERENCES stores(store_id),
    FOREIGN KEY (manager_id)
        REFERENCES employees(employee_id)
);
```

## 5.3 Customers Table

```
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL,
```

```
    email VARCHAR(120) UNIQUE,
    phone VARCHAR(15),
    city VARCHAR(50),
    registration_date DATE DEFAULT CURRENT_DATE,
    customer_type VARCHAR(20) DEFAULT 'Regular'
);
```

## 5.4 Categories Table

```
CREATE TABLE categories (
    category_id SERIAL PRIMARY KEY,
    category_name VARCHAR(80) NOT NULL UNIQUE,
    description VARCHAR(250)
);
```

## 5.5 Products Table

```
CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(120) NOT NULL,
    category_id INTEGER NOT NULL,
    price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    stock_quantity INTEGER DEFAULT 0 CHECK (stock_quantity >= 0),
    manufacturing_date DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id)
        REFERENCES categories(category_id)
);
```

## 5.6 Orders Table

```
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    employee_id INTEGER,
    store_id INTEGER,
    order_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    order_status VARCHAR(30) DEFAULT 'Pending',
    shipping_city VARCHAR(50),
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id),
    FOREIGN KEY (employee_id)
        REFERENCES employees(employee_id),
```

```
    FOREIGN KEY (store_id)
        REFERENCES stores(store_id)
);
```

### 5.7 Order Items Table

```
CREATE TABLE order_items (
    order_item_id SERIAL PRIMARY KEY,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(10,2) NOT NULL,
    discount_percentage NUMERIC(5,2) DEFAULT 0,
    FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE,
    FOREIGN KEY (product_id)
        REFERENCES products(product_id)
);
```

### 5.8 Payments Table

```
CREATE TABLE payments (
    payment_id SERIAL PRIMARY KEY,
    order_id INTEGER NOT NULL,
    payment_method VARCHAR(30),
    payment_amount NUMERIC(12,2) NOT NULL,
    payment_status VARCHAR(30),
    payment_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE
);
```

---

## 6. ALTER Table Operations

Add a column to the customers table:

```sql
ALTER TABLE customers
ADD COLUMN loyalty_points INTEGER DEFAULT 0;
```

Change the size of the phone column:

```sql
ALTER TABLE customers
ALTER COLUMN phone TYPE VARCHAR(20);
```

Rename a column:

```sql
ALTER TABLE stores
RENAME COLUMN active TO is_active;
```

Add a constraint:

```sql
ALTER TABLE employees
ADD CONSTRAINT employee_email_check
CHECK (email LIKE '%@%');
```

Drop a column:

```sql
ALTER TABLE customers
DROP COLUMN loyalty_points;
```

Add it again for later exercises:

```sql
ALTER TABLE customers
ADD COLUMN loyalty_points INTEGER DEFAULT 0;
```

---

# 7. Insert Sample Data

## 7.1 Stores

```sql
INSERT INTO stores
(store_name, city, state, opening_date, is_active)
VALUES
('SmartMart Pune', 'Pune', 'Maharashtra', '2020-01-15', TRUE),
```

```
('SmartMart Mumbai', 'Mumbai', 'Maharashtra', '2019-06-10', TRUE),
('SmartMart Bengaluru', 'Bengaluru', 'Karnataka', '2021-03-20', TRUE),
('SmartMart Hyderabad', 'Hyderabad', 'Telangana', '2022-07-01', TRUE),
('SmartMart Delhi', 'Delhi', 'Delhi', '2018-11-25', FALSE);
```

## 7.2 Employees

Insert managers first:

```
INSERT INTO employees
(employee_name, email, designation, salary, joining_date, store_id, manager_id)
VALUES
('Amit Sharma', 'amit@smartmart.com', 'Store Manager', 85000, '2019-01-10', 1,
NULL),
('Neha Verma', 'neha@smartmart.com', 'Store Manager', 90000, '2018-05-15', 2,
NULL),
('Rakesh Patil', 'rakesh@smartmart.com', 'Store Manager', 82000, '2020-02-01',
3, NULL),
('Priya Reddy', 'priya@smartmart.com', 'Store Manager', 80000, '2021-06-12', 4,
NULL);
```

Insert employees reporting to managers:

```
INSERT INTO employees
(employee_name, email, designation, salary, joining_date, store_id, manager_id)
VALUES
('Rahul Joshi', 'rahul@smartmart.com', 'Sales Executive', 45000, '2022-01-12',
1, 1),
('Sneha Kulkarni', 'sneha@smartmart.com', 'Sales Executive', 48000,
'2021-08-10', 1, 1),
('Vikas More', 'vikas@smartmart.com', 'Billing Executive', 42000, '2023-02-15',
2, 2),
('Pooja Shah', 'pooja@smartmart.com', 'Sales Executive', 50000, '2020-11-20',
2, 2),
('Kiran Rao', 'kiran@smartmart.com', 'Sales Executive', 47000, '2022-04-18', 3,
3),
('Anjali Singh', 'anjali@smartmart.com', 'Billing Executive', 44000,
'2023-07-05', 4, 4);
```

## 7.3 Customers

```
INSERT INTO customers
(customer_name, email, phone, city, registration_date, customer_type,
```

```
loyalty_points)
VALUES
('Arjun Mehta', 'arjun@gmail.com', '9876543210', 'Pune', '2023-01-10',
'Premium', 1200),
('Riya Kapoor', 'riya@gmail.com', '9876500011', 'Mumbai', '2023-02-15',
'Regular', 300),
('Sameer Khan', 'sameer@gmail.com', '9876500022', 'Pune', '2022-12-01',
'Premium', 900),
('Kavita Rao', 'kavita@gmail.com', '9876500033', 'Bengaluru', '2024-01-25',
'Regular', 150),
('Mohit Jain', 'mohit@gmail.com', '9876500044', 'Delhi', '2023-08-18',
'Regular', 200),
('Neha Patil', 'nehap@gmail.com', '9876500055', 'Pune', '2024-03-10',
'Premium', 750),
('Aditya Verma', 'aditya@gmail.com', '9876500066', 'Hyderabad', '2022-09-12',
'Corporate', 1500),
('Pallavi Joshi', 'pallavi@gmail.com', '9876500077', 'Mumbai', '2024-02-02',
'Regular', 100);
```

## 7.4 Categories

```
INSERT INTO categories
(category_name, description)
VALUES
('Laptops', 'Personal and business laptops'),
('Mobiles', 'Smartphones and mobile devices'),
('Accessories', 'Electronic accessories'),
('Home Appliances', 'Home and kitchen appliances'),
('Office Equipment', 'Equipment used in offices');
```

## 7.5 Products

```
INSERT INTO products
(product_name, category_id, price, stock_quantity, manufacturing_date)
VALUES
('Dell Inspiron Laptop', 1, 65000, 20, '2025-01-10'),
('HP Business Laptop', 1, 72000, 15, '2025-02-15'),
('Samsung Galaxy Phone', 2, 45000, 35, '2025-03-01'),
('Apple iPhone', 2, 85000, 12, '2025-01-20'),
('Wireless Mouse', 3, 1200, 100, '2025-04-05'),
('Mechanical Keyboard', 3, 3500, 60, '2025-02-25'),
('Microwave Oven', 4, 18000, 25, '2024-11-15'),
('Air Conditioner', 4, 42000, 10, '2025-01-05'),
```

```
('Office Printer', 5, 25000, 18, '2024-12-20'),
('Projector', 5, 55000, 8, '2025-03-12');
```

## 7.6 Orders

```
INSERT INTO orders
(customer_id, employee_id, store_id, order_timestamp, order_status,
shipping_city)
VALUES
(1, 5, 1, '2026-01-10 10:30:00', 'Completed', 'Pune'),
(2, 8, 2, '2026-01-11 12:15:00', 'Completed', 'Mumbai'),
(3, 6, 1, '2026-01-12 14:00:00', 'Pending', 'Pune'),
(4, 9, 3, '2026-01-14 16:45:00', 'Completed', 'Bengaluru'),
(5, 7, 2, '2026-01-15 11:20:00', 'Cancelled', 'Delhi'),
(6, 5, 1, '2026-01-16 18:10:00', 'Completed', 'Pune'),
(7, 10, 4, '2026-01-18 09:50:00', 'Pending', 'Hyderabad'),
(1, 6, 1, '2026-02-02 13:30:00', 'Completed', 'Pune');
```

## 7.7 Order Items

```
INSERT INTO order_items
(order_id, product_id, quantity, unit_price, discount_percentage)
VALUES
(1, 1, 1, 65000, 5),
(1, 5, 2, 1200, 0),
(2, 4, 1, 85000, 10),
(3, 6, 1, 3500, 0),
(4, 3, 2, 45000, 5),
(4, 5, 1, 1200, 0),
(5, 9, 1, 25000, 0),
(6, 7, 1, 18000, 8),
(6, 6, 1, 3500, 0),
(7, 8, 1, 42000, 5),
(8, 2, 1, 72000, 7);
```

## 7.8 Payments

```
INSERT INTO payments
(order_id, payment_method, payment_amount, payment_status, payment_timestamp)
VALUES
(1, 'Credit Card', 64150, 'Successful', '2026-01-10 10:35:00'),
(2, 'UPI', 76500, 'Successful', '2026-01-11 12:20:00'),
(4, 'Debit Card', 86700, 'Successful', '2026-01-14 16:50:00'),
```

```
(5, 'UPI', 25000, 'Refunded', '2026-01-15 11:25:00'),
(6, 'Net Banking', 20060, 'Successful', '2026-01-16 18:15:00'),
(8, 'Credit Card', 66960, 'Successful', '2026-02-02 13:35:00');
```

---

# 8. SQL Statement Fundamentals

## 8.1 Select All Records

```
SELECT * FROM customers;
```

## 8.2 Select Specific Columns

```
SELECT customer_name, email, city
FROM customers;
```

## 8.3 Using Column Aliases

```
SELECT
    customer_name AS customer,
    registration_date AS registered_on
FROM customers;
```

## 8.4 Ordering Records

```
SELECT product_name, price
FROM products
ORDER BY price DESC;
```

## 8.5 Limiting Results

```
SELECT product_name, price
FROM products
ORDER BY price DESC
LIMIT 5;
```

---

# 9. WHERE Clause Exercises

## 9.1 Equality Condition

```sql
SELECT *
FROM customers
WHERE city = 'Pune';
```

## 9.2 Multiple Conditions

```sql
SELECT *
FROM customers
WHERE city = 'Pune'
AND customer_type = 'Premium';
```

## 9.3 OR Condition

```sql
SELECT *
FROM customers
WHERE city = 'Pune'
OR city = 'Mumbai';
```

## 9.4 Comparison Operators

```sql
SELECT product_name, price
FROM products
WHERE price > 50000;
```

## 9.5 NOT Operator

```sql
SELECT *
FROM orders
WHERE order_status <> 'Completed';
```

---

# 10. DISTINCT

Display unique customer cities:

```
SELECT DISTINCT city
FROM customers;
```

Display unique order statuses:

```
SELECT DISTINCT order_status
FROM orders;
```

Count unique customer cities:

```
SELECT COUNT(DISTINCT city) AS total_customer_cities
FROM customers;
```

---

# 11. COUNT and Aggregate Functions

## 11.1 Count All Customers

```
SELECT COUNT(*) AS total_customers
FROM customers;
```

## 11.2 Count Completed Orders

```
SELECT COUNT(*) AS completed_orders
FROM orders
WHERE order_status = 'Completed';
```

## 11.3 Other Aggregate Functions

```
SELECT
    MIN(price) AS minimum_price,
    MAX(price) AS maximum_price,
    AVG(price) AS average_price,
```

```
    SUM(stock_quantity) AS total_stock
FROM products;
```

---

# 12. BETWEEN

Products priced between ₹20,000 and ₹70,000:

```
SELECT product_name, price
FROM products
WHERE price BETWEEN 20000 AND 70000;
```

Customers registered during 2023:

```
SELECT *
FROM customers
WHERE registration_date
BETWEEN '2023-01-01' AND '2023-12-31';
```

Orders placed within a date range:

```
SELECT *
FROM orders
WHERE order_timestamp
BETWEEN '2026-01-10 00:00:00'
AND '2026-01-16 23:59:59';
```

---

# 13. IN Operator

Find customers from selected cities:

```
SELECT *
FROM customers
WHERE city IN ('Pune', 'Mumbai', 'Bengaluru');
```

Find selected product categories:

```sql
SELECT *
FROM categories
WHERE category_name IN ('Laptops', 'Mobiles', 'Accessories');
```

Use NOT IN:

```sql
SELECT *
FROM orders
WHERE order_status NOT IN ('Cancelled', 'Pending');
```

---

# 14. LIKE and Pattern Matching

Customers whose names start with `A`:

```sql
SELECT *
FROM customers
WHERE customer_name LIKE 'A%';
```

Customers whose names end with `a`:

```sql
SELECT *
FROM customers
WHERE customer_name LIKE '%a';
```

Products containing the word `Laptop`:

```sql
SELECT *
FROM products
WHERE product_name LIKE '%Laptop%';
```

Case-insensitive search using PostgreSQL `ILIKE`:

```sql
SELECT *
FROM customers
WHERE customer_name ILIKE '%neha%';
```

---

# 15. GROUP BY

## 15.1 Count Customers by City

```sql
SELECT city, COUNT(*) AS total_customers
FROM customers
GROUP BY city;
```

## 15.2 Count Products by Category

```sql
SELECT
    category_id,
    COUNT(*) AS total_products
FROM products
GROUP BY category_id;
```

## 15.3 Average Salary by Store

```sql
SELECT
    store_id,
    ROUND(AVG(salary), 2) AS average_salary
FROM employees
GROUP BY store_id;
```

## 15.4 Orders by Status

```sql
SELECT
    order_status,
    COUNT(*) AS total_orders
FROM orders
GROUP BY order_status;
```

---

# 16. HAVING

Cities having more than one customer:

```sql
SELECT
    city,
    COUNT(*) AS total_customers
FROM customers
GROUP BY city
HAVING COUNT(*) > 1;
```

Stores with an average employee salary above ₹50,000:

```sql
SELECT
    store_id,
    AVG(salary) AS average_salary
FROM employees
GROUP BY store_id
HAVING AVG(salary) > 50000;
```

Products with total ordered quantity greater than one:

```sql
SELECT
    product_id,
    SUM(quantity) AS total_quantity
FROM order_items
GROUP BY product_id
HAVING SUM(quantity) > 1;
```

---

# 17. SQL JOIN Statements

## 17.1 INNER JOIN

Display products with their categories:

```sql
SELECT
    p.product_id,
    p.product_name,
    c.category_name,
    p.price
FROM products AS p
INNER JOIN categories AS c
    ON p.category_id = c.category_id;
```

Display orders with customer details:

```
SELECT
    o.order_id,
    c.customer_name,
    o.order_timestamp,
    o.order_status
FROM orders AS o
INNER JOIN customers AS c
    ON o.customer_id = c.customer_id;
```

## 17.2 LEFT JOIN

Display all customers, including customers without orders:

```
SELECT
    c.customer_id,
    c.customer_name,
    o.order_id,
    o.order_status
FROM customers AS c
LEFT JOIN orders AS o
    ON c.customer_id = o.customer_id;
```

Find customers who have never placed an order:

```
SELECT
    c.customer_id,
    c.customer_name
FROM customers AS c
LEFT JOIN orders AS o
    ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

## 17.3 RIGHT JOIN

Display all stores, including stores without employees:

```
SELECT
    s.store_name,
    e.employee_name,
    e.designation
```

```
FROM employees AS e
RIGHT JOIN stores AS s
    ON e.store_id = s.store_id;
```

### 17.4 FULL OUTER JOIN

Display all customers and all orders, including unmatched records:

```
SELECT
    c.customer_name,
    o.order_id,
    o.order_status
FROM customers AS c
FULL OUTER JOIN orders AS o
    ON c.customer_id = o.customer_id;
```

---

## 18. Multiple Table JOIN

Display complete order details:

```
SELECT
    o.order_id,
    c.customer_name,
    s.store_name,
    e.employee_name,
    p.product_name,
    oi.quantity,
    oi.unit_price
FROM orders AS o
INNER JOIN customers AS c
    ON o.customer_id = c.customer_id
INNER JOIN stores AS s
    ON o.store_id = s.store_id
LEFT JOIN employees AS e
    ON o.employee_id = e.employee_id
INNER JOIN order_items AS oi
    ON o.order_id = oi.order_id
INNER JOIN products AS p
    ON oi.product_id = p.product_id;
```

---

# 19. SELF JOIN

The employees table contains `manager_id`, which refers to another employee.

Display employees with their managers:

```
SELECT
    employee.employee_name AS employee,
    manager.employee_name AS manager
FROM employees AS employee
LEFT JOIN employees AS manager
    ON employee.manager_id = manager.employee_id;
```

Display employees earning more than their managers:

```
SELECT
    employee.employee_name AS employee,
    employee.salary AS employee_salary,
    manager.employee_name AS manager,
    manager.salary AS manager_salary
FROM employees AS employee
INNER JOIN employees AS manager
    ON employee.manager_id = manager.employee_id
WHERE employee.salary > manager.salary;
```

---

# 20. UNION

Combine customer cities and store cities:

```
SELECT city
FROM customers

UNION

SELECT city
FROM stores;
```

`UNION` removes duplicate values.

To retain duplicates:

```sql
SELECT city
FROM customers

UNION ALL

SELECT city
FROM stores;
```

Combine names of customers and employees:

```sql
SELECT
    customer_name AS person_name,
    'Customer' AS person_type
FROM customers

UNION

SELECT
    employee_name AS person_name,
    'Employee' AS person_type
FROM employees;
```

---

# 21. Subqueries

## 21.1 Products Above Average Price

```sql
SELECT product_name, price
FROM products
WHERE price > (
    SELECT AVG(price)
    FROM products
);
```

## 21.2 Employees Above Average Salary

```sql
SELECT employee_name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
```

```
    FROM employees
);
```

## 21.3 Customers Who Placed Orders

```
SELECT customer_name
FROM customers
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
);
```

## 21.4 Customers Without Orders

```
SELECT customer_name
FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id
    FROM orders
);
```

A safer version using `NOT EXISTS`:

```
SELECT c.customer_name
FROM customers AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.customer_id
);
```

## 21.5 Product with Maximum Price

```
SELECT product_name, price
FROM products
WHERE price = (
    SELECT MAX(price)
    FROM products
);
```

### 21.6 Correlated Subquery

Find employees earning more than the average salary of their store:

```
SELECT
    e.employee_name,
    e.store_id,
    e.salary
FROM employees AS e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees AS e2
    WHERE e2.store_id = e.store_id
);
```

---

## 22. Mathematical Functions

### 22.1 ROUND

```
SELECT
    product_name,
    price,
    ROUND(price * 1.18, 2) AS price_with_gst
FROM products;
```

### 22.2 CEIL and FLOOR

```
SELECT
    product_name,
    price,
    CEIL(price / 1000) AS upper_thousand,
    FLOOR(price / 1000) AS lower_thousand
FROM products;
```

### 22.3 ABS

```
SELECT ABS(-4500) AS absolute_value;
```

### 22.4 POWER

```
SELECT POWER(5, 2) AS result;
```

### 22.5 MOD

```
SELECT MOD(17, 5) AS remainder;
```

### 22.6 Random Number

```
SELECT RANDOM();
```

---

## 23. String Functions

### 23.1 UPPER and LOWER

```
SELECT
    UPPER(customer_name) AS uppercase_name,
    LOWER(email) AS lowercase_email
FROM customers;
```

### 23.2 LENGTH

```
SELECT
    customer_name,
    LENGTH(customer_name) AS name_length
FROM customers;
```

### 23.3 CONCAT

```
SELECT
    CONCAT(customer_name, ' - ', city) AS customer_details
FROM customers;
```

PostgreSQL concatenation operator:

```sql
SELECT
    customer_name || ' lives in ' || city AS customer_details
FROM customers;
```

## 23.4 SUBSTRING

```sql
SELECT
    customer_name,
    SUBSTRING(customer_name FROM 1 FOR 5) AS short_name
FROM customers;
```

## 23.5 TRIM

```sql
SELECT TRIM('   PostgreSQL Training   ') AS cleaned_text;
```

## 23.6 REPLACE

```sql
SELECT
    REPLACE(product_name, 'Laptop', 'Notebook') AS modified_name
FROM products;
```

## 23.7 POSITION

```sql
SELECT
    email,
    POSITION('@' IN email) AS at_symbol_position
FROM customers;
```

## 23.8 INITCAP

```sql
SELECT INITCAP('smart retail management system');
```

---

# 24. Date and Timestamp Functions

## 24.1 Current Date and Timestamp

```
SELECT CURRENT_DATE;
```

```
SELECT CURRENT_TIME;
```

```
SELECT CURRENT_TIMESTAMP;
```

```
SELECT NOW();
```

## 24.2 Extract Date Parts

```
SELECT
    order_id,
    EXTRACT(YEAR FROM order_timestamp) AS order_year,
    EXTRACT(MONTH FROM order_timestamp) AS order_month,
    EXTRACT(DAY FROM order_timestamp) AS order_day
FROM orders;
```

## 24.3 Calculate Employee Experience

```
SELECT
    employee_name,
    joining_date,
    AGE(CURRENT_DATE, joining_date) AS experience
FROM employees;
```

## 24.4 Add Time Interval

```
SELECT CURRENT_TIMESTAMP + INTERVAL '7 days';
```

Expected delivery date:

```sql
SELECT
    order_id,
    order_timestamp,
    order_timestamp + INTERVAL '5 days' AS expected_delivery
FROM orders;
```

### 24.5 Format Dates

```sql
SELECT
    order_id,
    TO_CHAR(order_timestamp, 'DD-MON-YYYY HH24:MI') AS formatted_order_time
FROM orders;
```

### 24.6 Date Truncation

Monthly order report:

```sql
SELECT
    DATE_TRUNC('month', order_timestamp) AS order_month,
    COUNT(*) AS total_orders
FROM orders
GROUP BY DATE_TRUNC('month', order_timestamp)
ORDER BY order_month;
```

---

## 25. Calculating Order Amount

Calculate amount before discount:

```sql
SELECT
    order_id,
    product_id,
    quantity,
    unit_price,
    quantity * unit_price AS gross_amount
FROM order_items;
```

Calculate amount after discount:

```sql
SELECT
    order_id,
    product_id,
    quantity,
    unit_price,
    discount_percentage,
    ROUND(
        quantity * unit_price *
        (1 - discount_percentage / 100),
        2
    ) AS net_amount
FROM order_items;
```

Calculate total order value:

```sql
SELECT
    order_id,
    ROUND(
        SUM(
            quantity * unit_price *
            (1 - discount_percentage / 100)
        ),
        2
    ) AS total_order_amount
FROM order_items
GROUP BY order_id;
```

---

# 26. UPDATE Statements

## 26.1 Update Customer Type

```sql
UPDATE customers
SET customer_type = 'Premium'
WHERE loyalty_points >= 700;
```

## 26.2 Increase Product Price

Increase laptop prices by 5%:

```sql
UPDATE products
SET price = price * 1.05
WHERE category_id = 1;
```

### 26.3 Update Order Status

```sql
UPDATE orders
SET order_status = 'Completed'
WHERE order_id = 3;
```

### 26.4 Update Multiple Columns

```sql
UPDATE customers
SET
    city = 'Navi Mumbai',
    phone = '9999999999'
WHERE customer_id = 2;
```

### 26.5 Update Using a Subquery

Give 100 additional loyalty points to customers who completed an order:

```sql
UPDATE customers
SET loyalty_points = loyalty_points + 100
WHERE customer_id IN (
    SELECT customer_id
    FROM orders
    WHERE order_status = 'Completed'
);
```

---

## 27. DELETE Statements

### 27.1 Delete One Record

```sql
DELETE FROM customers
WHERE customer_id = 8;
```

This operation may fail when the customer is referenced by an order.

### 27.2 Delete Cancelled Orders

```
DELETE FROM orders
WHERE order_status = 'Cancelled';
```

Because `order_items` and `payments` use `ON DELETE CASCADE`, their related records are automatically removed.

### 27.3 Delete Using a Condition

```
DELETE FROM products
WHERE stock_quantity = 0;
```

### 27.4 Delete All Rows

```
DELETE FROM payments;
```

This removes rows but retains the table structure.

---

## 28. DROP Statements

Drop a table:

```
DROP TABLE payments;
```

Prevent an error when the table does not exist:

```
DROP TABLE IF EXISTS payments;
```

Drop a table and dependent objects:

```
DROP TABLE categories CASCADE;
```

Use `CASCADE` carefully because it may remove dependent constraints and objects.

Drop a database:

```
DROP DATABASE smartmart_db;
```

The database cannot be dropped while connected to it.

---

# 29. TRUNCATE

Remove all records quickly:

```
TRUNCATE TABLE payments;
```

Remove data and reset identity values:

```
TRUNCATE TABLE payments
RESTART IDENTITY;
```

Truncate related tables:

```
TRUNCATE TABLE orders
RESTART IDENTITY
CASCADE;
```

---

# 30. Data Import Features

PostgreSQL supports importing data through:

- `COPY`
- `\copy`
- pgAdmin import utility
- `psql`
- `pg_restore`
- External ETL tools

## 30.1 Customer CSV File

Create a file named `customers.csv`:

```
customer_name,email,phone,city,registration_date,customer_type,loyalty_points
Raj Malhotra,raj@gmail.com,9000000001,Pune,2026-01-01,Regular,100
Simran Kaur,simran@gmail.com,9000000002,Mumbai,2026-01-02,Premium,800
Aman Gupta,aman@gmail.com,9000000003,Delhi,2026-01-03,Corporate,1200
```

## 30.2 Import Using COPY

The file must be accessible to the PostgreSQL server:

```
COPY customers
(
    customer_name,
    email,
    phone,
    city,
    registration_date,
    customer_type,
    loyalty_points
)
FROM '/tmp/customers.csv'
DELIMITER ','
CSV HEADER;
```

## 30.3 Import Using `\copy`

`\copy` reads the file from the client machine:

```
\copy
customers(customer_name,email,phone,city,registration_date,customer_type,loyalty_points)
FROM '/Users/username/customers.csv'
DELIMITER ','
CSV HEADER;
```

For Windows:

```
\copy
customers(customer_name,email,phone,city,registration_date,customer_type,loyalty_points)
FROM 'C:/training/customers.csv'
DELIMITER ','
CSV HEADER;
```

---

# 31. Data Export Features

## 31.1 Export Customers to CSV

```
COPY customers
TO '/tmp/customers_export.csv'
DELIMITER ','
CSV HEADER;
```

## 31.2 Export a Query Result

```
COPY (
    SELECT
        customer_name,
        email,
        city,
        customer_type
    FROM customers
    WHERE customer_type = 'Premium'
)
TO '/tmp/premium_customers.csv'
DELIMITER ','
CSV HEADER;
```

## 31.3 Export Using `\copy`

```
\copy (
    SELECT
        product_name,
        price,
        stock_quantity
    FROM products
)
TO '/Users/username/products_export.csv'
DELIMITER ','
CSV HEADER;
```

---

# 32. Database Backup and Restore

## 32.1 Backup Using pg_dump

```
pg_dump -U admin -d smartmart_db -F c -f smartmart_backup.dump
```

## 32.2 Export as SQL Script

```
pg_dump -U admin -d smartmart_db -f smartmart_backup.sql
```

## 32.3 Restore SQL Backup

```
psql -U admin -d smartmart_db -f smartmart_backup.sql
```

## 32.4 Restore Custom Format Backup

```
pg_restore -U admin -d smartmart_db smartmart_backup.dump
```

## 32.5 Backup from Docker Container

```
docker exec postgres-db pg_dump
  -U admin
  -d smartmart_db
  > smartmart_backup.sql
```

## 32.6 Restore into Docker PostgreSQL

```
docker exec -i postgres-db psql
  -U admin
  -d smartmart_db
  < smartmart_backup.sql
```

---

# 33. Case Study Exercise

Participants must complete the following tasks without directly copying the demonstrated queries.

## Exercise 1: Database Structure

1. Create the `smartmart_db` database.
2. Create all eight tables.
3. Add appropriate primary keys.
4. Add appropriate foreign keys.
5. Add `NOT NULL`, `UNIQUE`, `CHECK`, and `DEFAULT` constraints.
6. Create a self-referencing foreign key for employee managers.
7. Configure cascading deletion for order items and payments.

## Exercise 2: Table Modification

1. Add an `address` column to the customers table.
2. Add a `brand` column to the products table.
3. Change `customer_type` to `VARCHAR(30)`.
4. Rename `manufacturing_date` to `manufactured_on`.
5. Add a check constraint so discounts cannot exceed 50%.
6. Remove the `address` column.

## Exercise 3: Data Insertion

1. Insert at least five stores.
2. Insert at least fifteen employees.
3. Insert at least twenty customers.
4. Insert at least ten categories.
5. Insert at least thirty products.
6. Insert at least twenty orders.
7. Insert at least forty order items.
8. Insert payment information for completed orders.

## Exercise 4: Basic SELECT Queries

Write SQL queries to:

1. Display all products.
2. Display product name, price, and stock.
3. Display all active stores.
4. Display employees with salaries above ₹50,000.
5. Display customers from Pune.
6. Display customers who are not from Mumbai.
7. Display the five most expensive products.
8. Display products ordered alphabetically.

## Exercise 5: WHERE, BETWEEN, IN, and LIKE

Write SQL queries to:

1. Find products priced between ₹10,000 and ₹50,000.
2. Find customers from Pune, Mumbai, and Bengaluru.
3. Find completed and pending orders.
4. Find customers whose names start with `S`.
5. Find products containing the word `Phone`.
6. Find customers using a Gmail address.
7. Find employees who joined between 2020 and 2024.
8. Find products that are not in categories 1, 2, and 3.

## Exercise 6: DISTINCT and COUNT

Write SQL queries to:

1. Display unique customer cities.
2. Display unique customer types.
3. Count total products.
4. Count premium customers.
5. Count completed orders.
6. Count distinct cities where stores are located.
7. Count products with stock below ten.

## Exercise 7: GROUP BY and HAVING

Write SQL queries to:

1. Count customers by city.
2. Count employees by store.
3. Calculate average product price by category.
4. Calculate total stock by category.
5. Count orders by status.
6. Display cities with more than two customers.
7. Display categories whose average product price exceeds ₹30,000.
8. Display stores having more than three employees.

## Exercise 8: JOIN Operations

Write SQL queries to:

1. Display products with category names.
2. Display orders with customer names.
3. Display orders with customer and employee names.
4. Display employees with store names.

5. Display all customers, including those without orders.
6. Display all stores, including those without employees.
7. Display customers who have never placed an order.
8. Display complete order details with product names.
9. Display payment details with customer names.
10. Display orders with store, employee, customer, and payment information.

## Exercise 9: SELF JOIN

Write SQL queries to:

1. Display every employee with the manager's name.
2. Display employees who do not have a manager.
3. Display managers with the employees reporting to them.
4. Count the number of employees reporting to each manager.

## Exercise 10: UNION

Write SQL queries to:

1. Combine customer cities and store cities.
2. Combine customer names and employee names.
3. Display names with a column identifying whether each person is a customer or an employee.
4. Compare the results of `UNION` and `UNION ALL`.

## Exercise 11: Subqueries

Write SQL queries to:

1. Find products priced above the average product price.
2. Find employees earning above the average salary.
3. Find the most expensive product.
4. Find the second-most expensive product.
5. Find customers who placed at least one order.
6. Find customers who never placed an order.
7. Find products that have never been ordered.
8. Find employees earning above their store's average salary.
9. Find the customer who placed the highest number of orders.
10. Find the category with the highest average product price.

## Exercise 12: SQL Functions

Write SQL queries to:

1. Convert customer names to uppercase.
2. Convert product names to lowercase.

3. Display the length of every product name.
4. Combine customer name, email, and city into one column.
5. Replace `Laptop` with `Notebook` in product names.
6. Calculate product prices after adding 18% GST.
7. Round average product prices to two decimal places.
8. Display employee experience using the `AGE` function.
9. Display orders by month.
10. Calculate an expected delivery date five days after the order date.
11. Format timestamps as `DD-MM-YYYY HH24:MI`.
12. Display the current date and current timestamp.

## Exercise 13: UPDATE Operations

Write SQL statements to:

1. Increase all product prices by 10%.
2. Increase employee salaries by 5% for one store.
3. Mark a pending order as completed.
4. Add 500 loyalty points to premium customers.
5. Change customers with more than 1,000 points to `Platinum`.
6. Reduce stock after completing an order.
7. Update customer city and phone number together.

## Exercise 14: DELETE and DROP Operations

Write SQL statements to:

1. Delete a customer who has no orders.
2. Delete all cancelled orders.
3. Delete products having zero stock and no order history.
4. Remove all payment records using `DELETE`.
5. Compare `DELETE` and `TRUNCATE`.
6. Drop a temporary table.
7. Drop a table using `IF EXISTS`.

## Exercise 15: Import and Export

1. Create a CSV file containing ten customer records.
2. Import the CSV into PostgreSQL.
3. Export premium customers to CSV.
4. Export products with stock below ten.
5. Create a complete database backup.
6. Restore the backup into a new database named `smartmart_test_db`.

---

# 34. Final Reporting Challenge

Create the following management reports.

## Report 1: Monthly Sales Summary

Required columns:

- Month
- Number of orders
- Number of completed orders
- Total sales amount
- Average order amount

## Report 2: Store Performance

Required columns:

- Store name
- Number of employees
- Number of orders
- Total sales
- Average sales per order

## Report 3: Product Performance

Required columns:

- Product name
- Category
- Total quantity sold
- Total revenue
- Remaining stock

## Report 4: Customer Performance

Required columns:

- Customer name
- Customer type
- Total orders
- Total amount spent
- Last order date

### Report 5: Employee Sales Performance

Required columns:

- Employee name
- Store name
- Number of orders processed
- Total sales processed
- Average order value

### Report 6: Unsold Products

Display products that have never appeared in `order_items`.

### Report 7: Pending Payment Report

Display:

- Order ID
- Customer name
- Order amount
- Payment amount
- Pending amount

---

# 35. Sample Final Report Query

Store-wise sales performance:

```
SELECT
    s.store_name,
    COUNT(DISTINCT e.employee_id) AS total_employees,
    COUNT(DISTINCT o.order_id) AS total_orders,
    ROUND(
        COALESCE(
            SUM(
                oi.quantity *
                oi.unit_price *
                (1 - oi.discount_percentage / 100)
            ),
            0
        ),
        2
    ) AS total_sales
```

```sql
FROM stores AS s
LEFT JOIN employees AS e
    ON s.store_id = e.store_id
LEFT JOIN orders AS o
    ON s.store_id = o.store_id
    AND o.order_status = 'Completed'
LEFT JOIN order_items AS oi
    ON o.order_id = oi.order_id
GROUP BY
    s.store_id,
    s.store_name
ORDER BY total_sales DESC;
```

---

# 36. Expected Participant Deliverables

Each participant should submit:

1. Database creation SQL script
2. Table creation SQL script
3. Sample data insertion script
4. SQL query exercise file
5. CSV import file
6. CSV export files
7. PostgreSQL backup file
8. Screenshots of executed reports
9. A short explanation of table relationships
10. Final management report queries

---

# 37. Evaluation Criteria

| Area | Weightage |
|---|---|
| Database and table design | 15% |
| Primary and foreign keys | 10% |
| Insert, update, and delete operations | 10% |
| Basic SELECT queries | 10% |
| GROUP BY and HAVING | 10% |
| JOIN operations | 15% |

| Area | Weightage |
|---|---|
| Subqueries and UNION | 10% |
| Functions and timestamp operations | 10% |
| Import, export, backup, and restore | 5% |
| Final reporting challenge | 5% |

**Total: 100%**